Python type checking



Thomas Aglassinger, 2026-09-01, PyGraz

CC BY

Python typing

Dynamic typing (duck typing)

- No type declaration needed.
- Variables, parameters, return values can be any type.
- Variable can even change type during run-time.
- Errors show only during runtime.
- In case of type errors: Can be examined during runtime with print, debugger, etc.



Static typing

- Types need to be declared in the source code.
  - Type inference can be convenient:
    const name = "Alice" // derived type: String
- Types can be checked at build time, no surprises during runtime.
  (But: corner cases, reflection, types like "dynamic", ...)
- In case of type problems: The build error message is the only source of information.

Python type hints

Python type hints

- Variables, parameters and function results can have a type hint.
- Type hints are optional.
- **Types are never checked by Python.**
- The type hint is only an intention. The actual type can still be anything.
- Uses:
  - Documentation
  - Auto-complete in IDE
  - Quick navigation in source code

Basic examples

name: str

width: int

distance: float

is_ready: bool

Multiple types

width: int | float | decimal.Decimal

Optional values

width: int | None = None

Forward references

class **Person**:
    name
    home



class **Location**:
    owner
    ...



Forward references

class **Person**:
    name
    home

class **Location**:
    owner
    ...





Forward references

class **Person**:
    name
    home

class **Location**:
    owner
    ...





Forward references

```
class Person:
    name: str
    home

class Location:
    owner: Person
    ...
```

Forward references

```
class Person:
    name: str
    home: Location

class Location:
    owner: Person
    ...
```

Forward references

```
class Person:
    name: str
    home: "Location"

class Location:
    owner: Person
    ...
```

Optional forward references

```
class Person:
    name: str
    home: "Location" | None

class Location:
    owner: Person
    ...
```

Optional forward references

```
from typing import Optional

class Person:
    name: str
    home: Optional["Location"]

class Location:
    owner: Person
    ...
```

Lists

names: **list[str]** = ["Alice", "Bob", ...]

Similar: set, tuple

Dicts

```
pupil_to_grade_mapping: dict[str, int] = {
    "Alice": 2,
    "Bob": 3,
    "Claire": 2,
    ...
}
```

Dicts with optional entries

```
pupil_to_grade_mapping: dict[str, int | None] = {
    "Alice": 2,
    "Bob": 3,
    "Claire": None,  # absent
    ...
}
```

Generics / type variables

```
def doubled(a: int | float) -> int | float
    return a * 2


>>> doubled(3)
6  # int
>>> doubled (3.0)
6.0  # float
```

Generics / type variables

Confusion: When is the result int, and when is it float?

```
def doubled(a: int | float) -> int | float
    return a * 2
```

```
>>> doubled(3)
6  # int
>>> doubled (3.0)
6.0  # float
```

Generics / type variables

```
from typing import TypeVar

T = TypeVar("T")

def doubled(a: T) -> T:  # result type is the same as the parameter
    return a * 2


>>> doubled(3)
6  # int
>>> doubled (3.0)
6.0  # float
```

Generics / type variables

```
from typing import TypeVar

T = TypeVar("T")

def doubled(a: T) -> T:  # result type is the same as the parameter
    return a * 2


>>> doubled(decimal.Decimal("1.23"))
Decimal('2.46')  # Decimal
```

Generics / type variables

```
from typing import TypeVar

T = TypeVar("T")

def doubled(a: T) -> T:  # result type is the same as the parameter
    return a * 2


>>> doubled("hello")
"hellohello"  # str
```

Generics / type variables

```
from typing import TypeVar

T = TypeVar("T")

def doubled(a: T) -> T:  # result type is the same as the parameter
    return a * 2
```

```
>>> doubled("hello")
"hellohello"  # str
```



Constrained TypeVars

```
from typing import TypeVar

T = TypeVar("T", int, float)

def doubled(a: T) -> T:  # result type is the same as the parameter
    return a * 2
```

Type checker might still reject this

Generics bound to a class or subclass

```
from typing import TypeVar
from django.db import models

MODEL = TypeVar("MODEL", bound=models.Model)
```

Nightmare fuel: The "Generics" chapter in the Python docs
https://docs.python.org/3/library/typing.html#generics

Generator function

```
def factorials(n: int):
    result = 1
    for i in range(1, n + 1):
        result *= i
        yield result
```

Generator function

```
from collections.abc import Generator

def factorials(n: int) -> Generator[int, None, None]:
    result = 1
    for i in range(1, n + 1):
        result *= i
        yield result
```

Generator function

```
from collections.abc import Generator

def factorials(n: int) -> Generator[int, None, None]:
    result = 1
    for i in range(1, n + 1):
        result *= i
        yield result
```

Type of optional final "return"

Type accepted by .send()

Generator function

```
from collections.abc import Iterator

def factorials(n: int) -> Iterator[int]:
    result = 1
    for i in range(1, n + 1):
        result *= i
        yield result
```

Any

- represents "any type"
- Essentially disables type checking
- Function without result type hint returns Any



Any

- represents "any type"
- Essentially disables type checking
- Function without result type hint returns Any
- Need to specify None for functions without result

```
def something() -> None:
    ...
```



Caveats

- Type hints are still a moving target.
  - typing vs collections.abc
  - type vs TypeVar
  - tip: use ruff with pyupgrade rules: https://docs.astral.sh/ruff/rules/#pyupgrade-up
- Can be messy.
  - Callable (function hooks)
  - Lamda expressions
- Not everything runnable can be expressed as type hint.

Python type checking

Tools to type check

- ty: https://docs.astral.sh/ty/
  - Same people that made uv, ruff, prek
- pyrefly: https://pyrefly.org/
  - Default in PyCharm
  - Same people that made Facebook
- mypy: https://www.mypy-lang.org/
  - Grand daddy of Python type checking

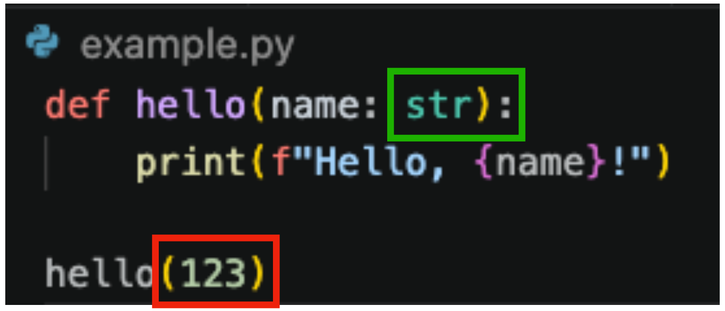
Running ty

example.py

```python
def hello(name: str):
    print(f"Hello, {name}!")

hello(123)
```

Running ty



Running ty

example.py

```python
def hello(name: str):
    print(f"Hello, {name}!")

hello(123)
```

```
Installed 1 package in 2ms
error[invalid-argument-type]: Argument to function `hello` is incorrect
 --> /tmp/example.py:4:7
  |
4 | hello(123)
  |       ^^^ Expected `str`, found `Literal[123]`
  |
info: Function defined here
 --> /tmp/example.py:1:5
  |
1 | def hello(name: str):
  |     ^^^^^ --------- Parameter declared here
  |
```

ty rule configuration example

pyproject.toml

```
[tool.ty.rules]
unused-ignore-comment = "warn"
redundant-cast = "ignore"
possibly-missing-attribute = "error"
```

ty suppression comments



Personal experience

- ✅ Easy to add to project.
- ✅ Decent integration in IDEs like PyCharm.
- ❌ Does not work well with Pydantic or Django.
- ❌ Error message in terminal not clickable for PyCharm.

pre-commit check

ty pre-commit check

Use local uvx

```
- repo: local
    hooks:
      - id: ty
        name: ty
        entry: uvx ty check
        language: system
        types: [python]
        pass_filenames: false
```

Conclusion

Personal recommendation (for now)

- Use type hints for basic types
  - ...especially to document what can be None
- Use type hints for classes
- Use type hints for complex typing only if you know how to phrase them
- Use **type checkers** only if they do not cause too much trouble
  (e.g. avoid with Django)

Python type checking

- Python type hints still have their quirks.
- With the right tools, Python type hints can be used for type checks.

